

Clasificación de imágenes de alimentos con Transformers

Benchmark sobre Food-101

Informe técnico: Trabajo integrador

Maestría en Inteligencia Artificial - FIUBA

2026

Visión por Computadora III

Benchmark de *Vision Transformer* sobre dataset Food-101 para una aplicación móvil

Informe técnico

Antonella Nerea Gambarte, Florencia Priscilla Vela, Marcos Lund

[Repositorio del proyecto](#)

Resumen. El objetivo de este trabajo es comparar distintas arquitecturas *Vision Transformer* (MobileViT-S, DeiT-Ti, Swin-T y ViT-B/16), poniendo especial énfasis en las de menor costo computacional, MobileViT-S y DeiT-Ti, que podrían resultar adecuadas para escenarios con recursos limitados. La comparación busca analizar el equilibrio entre desempeño de clasificación y costo computacional, con el objetivo de identificar una arquitectura adecuada para una posible aplicación móvil de clasificación de alimentos.

Los cuatro modelos se evaluaron sobre Food-101 mediante un protocolo común de *fine-tuning* y evaluación, complementado posteriormente con pruebas de ajuste sobre algunas arquitecturas. ViT-B/16 alcanzó el mayor desempeño con 88,36 % de *accuracy*. MobileViT-S obtuvo un desempeño cercano, con 86,05 %, utilizando aproximadamente 17 veces menos parámetros y casi 9 veces menos *GFLOPs* que ViT-B/16. Swin-T presentó resultados próximos a MobileViT-S, aunque con un costo computacional mayor, mientras que DeiT-Ti mostró el menor desempeño de clasificación. En conjunto, los resultados muestran que MobileViT-S ofrece una relación favorable entre desempeño y eficiencia para escenarios con recursos limitados, mientras que ViT-B/16 obtiene el mejor desempeño cuando se prioriza la capacidad de clasificación.

1. Objetivo del proyecto

El objetivo es estudiar qué arquitectura *Vision Transformer* ofrece el mejor equilibrio entre capacidad de clasificación y costo computacional para una aplicación que identifique platos de comida a partir de una fotografía, pensando especialmente en su posible uso en dispositivos con recursos limitados.

Las preguntas principales son:

- ¿Cuánto desempeño se pierde (si se pierde) al utilizar una arquitectura liviana en lugar de una referencia de mayor capacidad?
- ¿La diferencia entre modelos se concentra en las clases visualmente más difíciles o se distribuye de manera uniforme?
- ¿Qué costo en parámetros, operaciones, tamaño de pesos y latencia acompaña a cada nivel de desempeño?

2. Dataset y análisis exploratorio

2.1 Food-101

Se utiliza Food-101, un *dataset* de clasificación de alimentos compuesto por 101.000 imágenes distribuidas uniformemente en 101 clases, con 1.000 imágenes por clase. La partición oficial asigna 750 imágenes por clase al conjunto de entrenamiento y 250 al conjunto de *test*. Las imágenes fueron obtenidas originalmente de Foodspotting.com, donde eran publicadas y etiquetadas por los usuarios, por lo que algunas podían contener etiquetas incorrectas. Los autores de Food-101 revisaron manualmente las imágenes destinadas al conjunto de *test*, mientras que mantuvieron el conjunto de entrenamiento sin limpiar. Por este motivo, este último conserva cierto nivel de ruido y algunas etiquetas incorrectas.

En este proyecto se conservó el conjunto de *test* oficial como conjunto final de evaluación. A partir de la partición oficial de entrenamiento se generaron posteriormente los conjuntos de entrenamiento y validación utilizados durante el ajuste de los modelos.

Característica	Cantidad
Clases	101
Imágenes totales	101.000
Train oficial	75.750 imágenes
Test oficial	25.250 imágenes
Imágenes por clase	1.000

2.2 Decisiones derivadas del EDA

El EDA se realizó con el objetivo de analizar las principales características del dataset y utilizar sus resultados para definir decisiones posteriores del pipeline. Se analizaron la distribución de clases, la geometría de las imágenes, la integridad y el formato de los archivos, la presencia de duplicados y la dificultad relativa de las clases.

La distribución por clase confirmó que el dataset se encuentra balanceado, por lo que no fue necesario incorporar estrategias específicas para compensar desbalance durante el entrenamiento.

El análisis de la geometría mostró que las imágenes varían en tamaño y relación de aspecto, y que cada arquitectura espera una entrada distinta (otra resolución, otro recorte, otra normalización). Por eso se decidió preparar las imágenes de cada modelo tal como las espera su checkpoint, en lugar de fijar un preprocesamiento común. Ese preprocesamiento lo define el `AutoImageProcessor` de cada modelo. Cómo se reproduce en el pipeline (y por qué no se usa ese objeto directamente) se detalla en la sección 4.1.

Para analizar la integridad de los datos se inspeccionó una muestra estratificada de 6.060 imágenes. En esta muestra no se detectaron archivos corruptos ni imágenes con un modo de color distinto de RGB. Como la inspección no abarcó la totalidad del dataset, el pipeline realiza igualmente la conversión a RGB de manera preventiva para garantizar una entrada consistente de tres canales.

También se analizaron posibles duplicados y fugas entre particiones, sin encontrarse problemas relevantes en las imágenes evaluadas.

Finalmente, se estimó la dificultad relativa de las clases para analizar si los modelos se comportan de manera diferente según el nivel de dificultad. Para ello se generó un ranking por clase y se definió un subconjunto fijo de 2.525 imágenes, con 25 ejemplos por clase, que se utilizó para comparar todas las arquitecturas sobre exactamente las mismas imágenes.

3. Arquitectura de los modelos

3.1 Descripción general

En el proyecto se evaluaron cuatro arquitecturas basadas en *Transformers* para clasificación de imágenes: ViT-B/16, DeiT-Ti, Swin-T y MobileViT-S. La selección se realizó de acuerdo con el objetivo del trabajo, buscando comparar modelos con distintos niveles de complejidad y costo computacional, desde arquitecturas más livianas y orientadas a entornos con recursos limitados hasta una referencia de mayor capacidad.

Modelo	Checkpoint	Parámetros
MobileViT-S	apple/mobilevit-small	5,00 M
DeiT-Ti	facebook/deit-tiny-patch16-224	5,54 M
Swin-T	microsoft/swin-tiny-patch4-window7-224	27,60 M
ViT-B/16	google/vit-base-patch16-224-in21k	85,88 M

3.2 ViT-B/16

Vision Transformer (ViT) adapta la arquitectura *Transformer* al procesamiento de imágenes. La imagen se divide en parches de tamaño fijo y cada uno se convierte en una

representación vectorial que pasa a formar parte de una secuencia. En ViT-B/16, estos parches tienen un tamaño de 16x16 píxeles.

La secuencia completa es procesada mediante bloques *Transformer Encoder* con *self-attention* global, lo que permite que cada parche pueda relacionarse directamente con todos los demás.

Componentes clave

- *Patch embedding*: transforma cada parche de la imagen en una representación vectorial.
- *Embeddings* posicionales: incorporan información sobre la posición original de cada parche.
- *Token* de clasificación [CLS]: token especial agregado a la secuencia cuya representación final se utiliza para realizar la clasificación.
- Bloques *Transformer Encoder*: procesan la secuencia mediante *self-attention* y capas *feed-forward*.
- Cabeza de clasificación: transforma la representación final en las probabilidades correspondientes a las 101 clases.

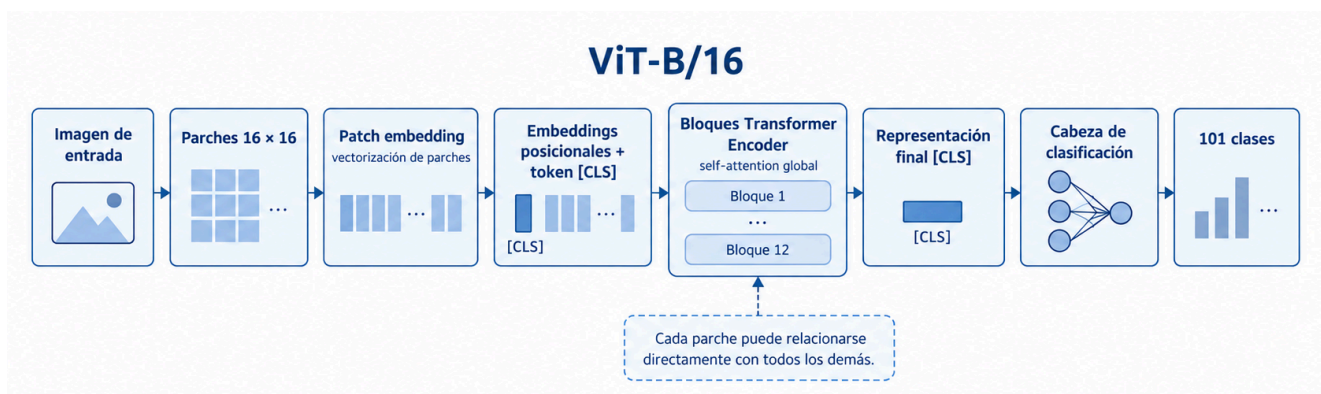


Figura 1. Arquitectura de ViT-B/16.

3.3 DeiT-Ti

DeiT (*Data-efficient Image Transformer*) parte de la arquitectura de *Vision Transformer* (ViT), pero fue entrenado mediante una estrategia más eficiente en el uso de datos, orientada a obtener buenos resultados con una cantidad de datos y recursos de entrenamiento menor. Al igual que ViT, divide la imagen en parches de tamaño fijo, transforma cada parche en una representación vectorial y procesa la secuencia mediante bloques *Transformer Encoder*. También incorpora *embeddings* posicionales y un *token* de clasificación [CLS], cuya representación final se utiliza para realizar la predicción.

Dentro de la familia DeiT existen distintas variantes que difieren principalmente en su tamaño y en el uso o no de destilación. En este proyecto se utilizó DeiT-Tiny, la variante más compacta de la familia y sin destilación, lo que permite incorporar al *benchmark* un *Vision Transformer* puro de tamaño comparable al de MobileViT-S.

Componentes clave

- *Patch embedding*: transforma los parches de la imagen en representaciones vectoriales.
- *Embeddings* posicionales: conservan información sobre la ubicación de cada parche.
- *Token* de clasificación [CLS]: token especial agregado a la secuencia cuya representación final se utiliza para realizar la clasificación.
- Bloques *Transformer Encoder*: aplican *self-attention* y capas *feed-forward* sobre la secuencia.
- Cabeza de clasificación: produce la predicción final.

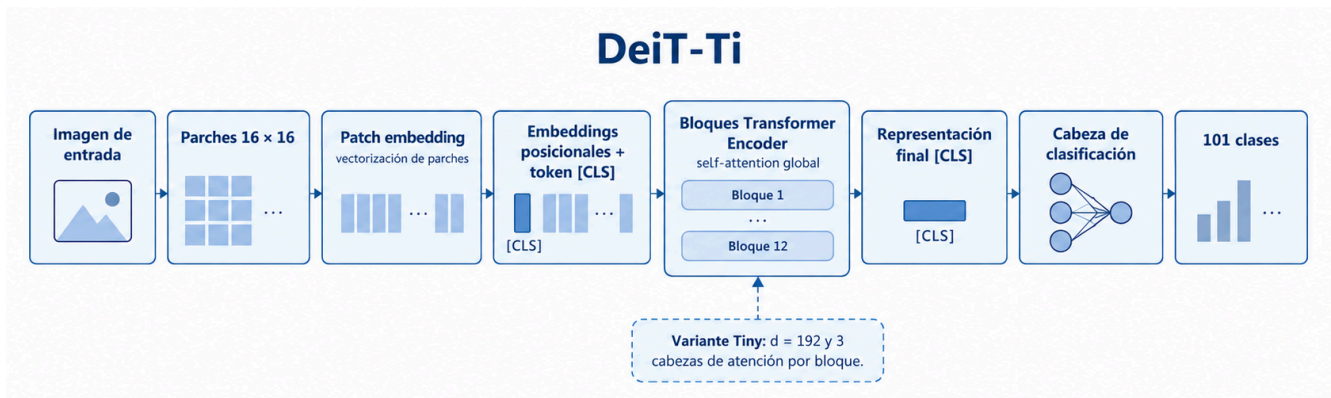


Figura 2. Arquitectura de DeiT-Ti.

3.4 Swin-T

Swin Transformer utiliza una arquitectura jerárquica. En lugar de aplicar atención global sobre todos los parches de la imagen, calcula *self-attention* dentro de ventanas locales.

Entre bloques consecutivos, estas ventanas se desplazan mediante el mecanismo de *shifted windows*, permitiendo que regiones que inicialmente pertenecían a ventanas diferentes intercambien información. A medida que avanza la red, los parches se combinan y la resolución espacial disminuye, generando representaciones a distintas escalas.

Componentes clave

- *Patch embedding*: genera las representaciones iniciales de los parches de la imagen.
- Bloques *Swin Transformer*: aplican atención dentro de ventanas locales.
- *Shifted windows*: desplazan las ventanas entre bloques para permitir el intercambio de información entre regiones vecinas.
- Capas de *patch merging*: combinan parches vecinos y reducen progresivamente la resolución espacial.
- Cabeza de clasificación: utiliza la representación final para obtener la predicción de clase.

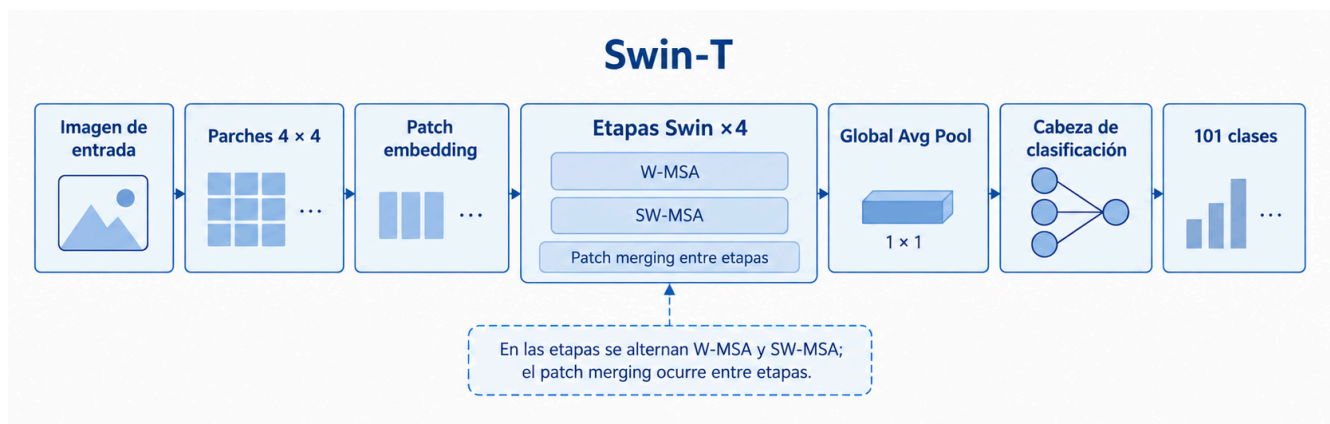


Figura 3. Arquitectura de Swin-T.

3.5 MobileViT-S

MobileViT es una arquitectura híbrida que combina convoluciones con bloques *Transformer*. Las capas convolucionales permiten capturar características locales de la imagen, mientras que los bloques MobileViT incorporan información global entre regiones más alejadas.

Dentro de los bloques MobileViT, los mapas de características se reorganizan en pequeños parches que son procesados mediante un *Transformer*. Luego, esa información se reconstruye y se combina nuevamente con las características locales obtenidas mediante convoluciones.

Componentes clave

- Capas convolucionales y bloques *MobileNetV2*: extraen características locales y reducen progresivamente la resolución espacial.
- Bloques MobileViT: combinan procesamiento convolucional local con información global obtenida mediante *Transformers*.
- Bloques *Transformer* internos: procesan los parches y modelan relaciones entre regiones alejadas.
- *Pooling* global: condensa la representación final de la imagen.
- Cabeza de clasificación: produce la predicción correspondiente a una de las 101 clases.

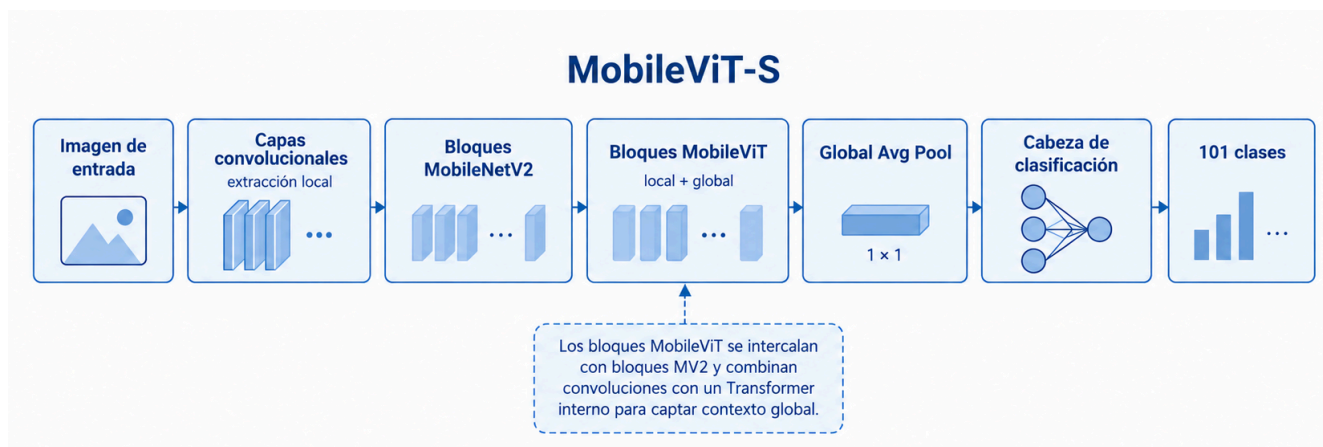


Figura 4. Arquitectura de MobileViT-S.

4. Implementación técnica

4.1 Preprocesamiento y *augmentation*

El pipeline obtiene, del AutoImageProcessor de cada modelo, la configuración de su entrada: la resolución objetivo, si recorta o no, los parámetros de normalización y el orden de los canales. Con esa configuración se construyen dos políticas de transformación.

La política de evaluación replica exactamente el preprocesamiento del AutoImageProcessor de cada *checkpoint* y se usa en validación y en *test*, sin *data augmentation*. Así, todos los modelos se evalúan bajo las condiciones con las que fueron preentrenados, y las diferencias medidas no dependen de la preparación de las imágenes. Que la réplica es exacta se verifica comparando, sobre una misma imagen, el tensor producido por el AutoImageProcessor con el generado por nuestra réplica: ambos coinciden hasta el límite de precisión de los números float32, es decir, la única diferencia es el redondeo inevitable de las operaciones y no un cambio real en la imagen.

La política de entrenamiento agrega *data augmentation* para aumentar la variedad de ejemplos que ve el modelo. Esta decisión es especialmente relevante en Food-101, donde las imágenes de una misma clase varían mucho en punto de vista, iluminación, tamaño y disposición del alimento. Se compone de dos transformaciones.

- *RandomResizedCrop*, con un rango de escala entre 0,65 y 1,0, presenta en cada época distintos encuadres y escalas de la misma fotografía, de modo que la posición y el tamaño exactos del plato no determinen la predicción. El rango es conservador para no eliminar partes relevantes del alimento.
- *RandomHorizontalFlip* refleja aleatoriamente la imagen en sentido horizontal, lo que no cambia el alimento presente pero reduce la dependencia respecto de posiciones izquierda/derecha accidentales. Después de la geometría aleatoria se aplica la misma cola de reescalado, normalización y orden de canales que en evaluación, derivada del *checkpoint* de cada modelo.

La misma política de entrenamiento se usó en todas las corridas y para las cuatro arquitecturas. Lo que varió entre corridas fue el *learning rate* y el tope de épocas (sección 6.4), nunca el preprocesamiento ni la *augmentation*.

4.2 Configuración de entrenamiento común

Hiperparámetro	Valor base
Máximo de épocas	20
<i>Learning rate</i>	5×10^{-4}
<i>Weight decay</i>	0,01
<i>Warmup ratio</i>	0,05
<i>Batch</i> efectivo	32
<i>Early stopping</i>	Paciencia 3
Métrica de selección	F1 macro

Se definió inicialmente una configuración común de *fine-tuning* para las cuatro arquitecturas, con el objetivo de realizar una primera comparación bajo condiciones de entrenamiento equivalentes. Como punto de partida se utilizó un *learning rate* de 5×10^{-4} y un máximo de 20 épocas, junto con *early stopping* basado en F1 macro de validación (ver 5.1 sobre la elección de esta métrica).

El *batch* efectivo se mantuvo en 32 imágenes. Para adaptar el *batch* físico a la memoria disponible sin modificar esta condición entre arquitecturas, se utilizó acumulación de gradientes cuando fue necesario. De esta manera, el modelo puede procesar una cantidad menor de imágenes simultáneamente y acumular sus gradientes antes de realizar la actualización de los pesos, manteniendo un *batch* efectivo común.

4.3 Herramientas

La implementación utiliza Python, PyTorch, Hugging Face Transformers, torchvision y scikit-learn. El entorno y las dependencias se gestionan con uv a partir de pyproject.toml. Durante el entrenamiento, las métricas se registran en TensorBoard para visualizar su evolución, mientras que MLflow se utiliza para centralizar y comparar las distintas corridas a partir de los resultados almacenados en /reports/results.

5. Evaluación

5.1 Métricas de desempeño

Se reporta la *top-1 accuracy*: dado que Food-101 está balanceado, mide de forma directa el porcentaje de imágenes clasificadas correctamente y es comparable entre clases y entre modelos.

Durante el entrenamiento, en cambio, la métrica de selección del mejor *checkpoint* y de *early stopping* fue el F1 macro. Esa elección viene del diseño original del proyecto, que apuntaba a otro dataset, desbalanceado, donde la *accuracy* no es una métrica adecuada. El dataset se cambió a Food-101 para demostrar mejor las comparaciones entre modelos, y se mantuvo la

receta de entrenamiento. Como Food-101 está balanceado, ambas métricas se mueven juntas: en las ocho corridas realizadas, la *accuracy* y el F1 macro sobre el *test* difieren en menos de 0,1 punto, de modo que reportar *accuracy* no pierde información.

Además, se analiza el desempeño sobre los terciles de dificultad definidos durante el EDA, con el objetivo de observar si las diferencias entre arquitecturas se concentran en las clases fáciles, medias o difíciles.

5.2 Métricas de eficiencia

Además de las métricas de clasificación, se evaluó el costo computacional de cada arquitectura, ya que el proyecto considera como caso de uso la ejecución en dispositivos con recursos limitados. Para ello se analizaron la cantidad de parámetros, los GFLOPs, el tamaño de los pesos y la latencia de inferencia.

El tamaño de los pesos se estimó para precisiones FP32, FP16 e INT8 a partir de la cantidad de parámetros. Adicionalmente, se generó una versión cuantizada a INT8 mediante cuantización dinámica de las capas `nn.Linear`, sobre la cual se midieron el tamaño real obtenido y la latencia en CPU.

Para que la comparación de latencia entre arquitecturas fuera consistente, las mediciones se realizaron en el mismo equipo y bajo las mismas condiciones. Se utilizó *batch* 1, representando un escenario en el que se clasifica una imagen por vez, y las mediciones en CPU se realizaron utilizando un único hilo.

Estas mediciones permiten comparar las arquitecturas entre sí, aunque no equivalen a una evaluación directa del rendimiento en un dispositivo móvil real.

6. Resultados

En una primera etapa se realizó una comparación de las cuatro arquitecturas utilizando la configuración de entrenamiento común definida previamente. Los resultados presentados a continuación corresponden a estas primeras corridas y permiten establecer un punto de partida para comparar el desempeño y la eficiencia de los modelos. A partir de estos resultados se realizaron posteriormente nuevas pruebas y ajustes sobre algunas de las arquitecturas.

6.1 Resultados globales sobre el test oficial

Modelo	<i>Accuracy</i>	GFLOPs	CPU int8 mediana
MobileViT-S	86,05 %	4,00	74,5 ms
Swin-T	85,48 %	8,98	89,1 ms
ViT-B/16	82,39 %	35,13	200,7 ms
DeiT-Ti	77,68 %	2,51	19,0 ms

La siguiente tabla resume la cantidad de parámetros y el tamaño de los pesos de cada arquitectura:

Modelo	Parámetros	Tamaño FP32	Tamaño INT8 medido
MobileViT-S	5,00 M	19,08 MB	10,85 MB
Swin-T	27,60 M	105,27 MB	26,72 MB
ViT-B/16	85,88 M	327,59 MB	84,49 MB
DeiT-Ti	5,54 M	21,15 MB	6,02 MB

La cuantización dinámica se aplicó únicamente a las capas nn.Linear. Por este motivo, en ViT-B/16, DeiT-Ti y Swin-T, donde una gran proporción de los parámetros se encuentra en capas lineales, el tamaño medido en INT8 se reduce aproximadamente a una cuarta parte del tamaño FP32. En MobileViT-S, en cambio, las capas convolucionales no son afectadas por esta cuantización y permanecen en su precisión original, por lo que la reducción de tamaño es menor.

En esta primera comparación, MobileViT-S obtuvo el mayor desempeño global, con una *accuracy* de 86,05 %. Swin-T presentó resultados cercanos, con 85,48 %, aunque utilizando alrededor de 5,5 veces más parámetros y más del doble de GFLOPs que MobileViT-S.

ViT-B/16 presentó el mayor costo computacional de las arquitecturas evaluadas, tanto en cantidad de parámetros como en GFLOPs, sin alcanzar en esta primera prueba el desempeño de MobileViT-S y Swin-T. Por otro lado, DeiT-Ti fue el modelo con menor cantidad de GFLOPs y menor latencia CPU INT8, aunque también obtuvo la menor *accuracy*.

Estos resultados muestran que, bajo la configuración de entrenamiento utilizada en esta primera etapa, existe un compromiso entre desempeño y eficiencia: una mayor cantidad de parámetros o un mayor costo computacional no se tradujo necesariamente en un mejor resultado de clasificación.

6.2 Resultados por dificultad

Modelo	Fácil	Medio	Diffcil
MobileViT-S	92,82 %	87,52 %	78,24 %
Swin-T	92,47 %	86,18 %	78,82 %
ViT-B/16	90,82 %	83,27 %	74,24 %
DeiT-Ti	87,76 %	78,18 %	70,47 %

Para analizar el comportamiento de los modelos según la dificultad de las clases, se utilizó el subconjunto fijo de 2.525 imágenes definido durante el EDA, compuesto por 25 imágenes de cada una de las 101 clases y agrupado en terciles de dificultad. La tabla presenta la *accuracy* obtenida en cada grupo y corresponde a la primera comparación realizada con la configuración de entrenamiento común.

En los cuatro modelos se observa una disminución de la *accuracy* al pasar de las clases fáciles a las difíciles, consistente con la clasificación de dificultad establecida durante el EDA. MobileViT-S y Swin-T presentaron resultados muy cercanos: MobileViT-S obtuvo una mayor *accuracy* en los grupos fácil y medio, mientras que en el grupo difícil ambos modelos

quedaron prácticamente empatados (78,82 % para Swin-T y 78,24 % para MobileViT-S). ViT-B/16 y DeiT-Ti mostraron un desempeño menor en los tres grupos.

6.3 Errores compartidos

En esta primera comparación, el 68,1 % de las imágenes del *test* fue clasificado correctamente por los cuatro modelos, mientras que aproximadamente el 5,7 % fue clasificado incorrectamente por todos. Además, en 331 imágenes los cuatro modelos coincidieron incluso en la misma clase equivocada.

Con las configuraciones finalmente seleccionadas (sección 6.4), los cuatro modelos aciertan el 69,0 % de las imágenes y fallan en el 4,5 % (1.133 imágenes). En 280 de esos casos, además, los cuatro predicen la misma clase incorrecta. Estos errores compartidos son más frecuentes en las clases difíciles: representan el 7,2 % de las imágenes de ese grupo, frente al 2,5 % en las clases fáciles.

De manera complementaria, al analizar las confusiones más frecuentes de cada modelo se observan pares de clases similares, como *steak / filet mignon*, *chocolate cake / chocolate mousse* y *tuna tartare / beef tartare*. Estos pares aparecen entre los errores de las distintas arquitecturas, aunque con diferente frecuencia.

Por ejemplo, de las 250 imágenes de *filet mignon*, MobileViT-S clasificó 50 como *steak* (20,0 %), ViT-B/16 clasificó 35 (14,0 %), Swin-T 33 (13,2 %) y DeiT-Ti 46 (18,4 %). Para *chocolate cake*, las imágenes clasificadas erróneamente como *chocolate mousse* fueron 32 en MobileViT-S (12,8 %), 30 en ViT-B/16 (12,0 %), 35 en Swin-T (14,0 %) y 33 en DeiT-Ti (13,2 %). De manera similar, para *tuna tartare*, las predicciones incorrectas como *beef tartare* fueron 26 (10,4 %), 27 (10,8 %), 17 (6,8 %) y 28 (11,2 %), respectivamente.

Con el ViT-B/16 ajustado (sección 6.4), 55 de las 250 imágenes de *filet mignon* fueron clasificadas como *steak* (22,0 %), la frecuencia más alta de esta confusión entre los cuatro modelos. Aunque el ajuste mejoró el desempeño global de ViT-B/16, esta confusión continuó apareciendo, al igual que en las demás arquitecturas.

Estos resultados muestran que existen patrones de confusión que se repiten entre las arquitecturas, aunque la frecuencia con la que se produce cada error varía entre los modelos.

6.4 Pruebas adicionales de ajuste

Los resultados de la primera comparación mostraron que la configuración común de entrenamiento no resultaba igualmente adecuada para todas las arquitecturas. A partir de este análisis se realizaron nuevas corridas variando el *learning rate* o el número máximo de épocas para estudiar su efecto sobre el desempeño.

Modelo	Ajuste realizado	Épocas entrenadas	Accuracy test
Swin-T	30 épocas, LR 5×10^{-4}	30	85,52 %
MobileViT-S	LR 5×10^{-5} , 20 épocas	20	82,15 %

ViT-B/16	LR 5×10^{-5} , 20 épocas máx.	8	88,36 %
MobileViT-S	LR 5×10^{-5} , 30 épocas máx.	24	83,31 %

En Swin-T se amplió el máximo de 20 a 30 épocas porque al finalizar la corrida inicial la *accuracy* de validación todavía mostraba una tendencia de mejora. Sin embargo, la nueva configuración no mejoró la mejor *accuracy* de validación, que pasó de 81,53 % a 80,91 %. En el *test*, la *accuracy* se mantuvo prácticamente sin cambios, pasando de 85,48 % a 85,52 %.

En ViT-B/16 se evaluó un *learning rate* de 5×10^{-5} como alternativa a la configuración común de 5×10^{-4} , con el objetivo de analizar si actualizaciones más pequeñas favorecían el *fine-tuning* de esta arquitectura. Con el nuevo valor, la mejor *accuracy* de validación aumentó de 77,08 % a 84,32 %. Esta mejora también se observó sobre el *test*, donde la *accuracy* aumentó de 82,39 % a 88,36 %. El mejor valor de validación se alcanzó en la época 5 y el *early stopping* detuvo el entrenamiento en la época 8.

Para MobileViT-S también se evaluó un *learning rate* de 5×10^{-5} para comprobar si la arquitectura se beneficiaba de la misma reducción. En esta prueba, la mejor *accuracy* de validación disminuyó de 81,64 % a 77,03 %, mientras que la *accuracy* sobre *test* pasó de 86,05 % a 82,15 %. Dado que el menor *learning rate* implicaba un aprendizaje más gradual, se realizó una segunda corrida aumentando el máximo a 30 épocas. El entrenamiento se detuvo en la época 24 y alcanzó una *accuracy* de validación de 78,30 % y una *accuracy* de *test* de 83,31 %, resultados todavía inferiores a los obtenidos con la configuración original.

El efecto del ajuste de ViT-B/16 también se observó al analizar las clases por dificultad. La *accuracy* aumentó de 90,82 % a 94,47 % en las clases fáciles, de 83,27 % a 90,18 % en las medias y de 74,24 % a 82,24 % en las difíciles. La mejora fue especialmente marcada en este último grupo.

Estas pruebas muestran que las arquitecturas no responden de la misma manera a una configuración de entrenamiento común. Entre las configuraciones evaluadas, MobileViT-S obtuvo su mejor resultado con el *learning rate* inicial de 5×10^{-4} , mientras que ViT-B/16 mejoró sustancialmente al utilizar un valor diez veces menor.

Con las configuraciones seleccionadas, ViT-B/16 obtuvo el mejor desempeño sobre el *test* completo, superando a MobileViT-S por 2,3 puntos de *accuracy*, a Swin-T por 2,9 y a DeiT-Ti por 10,7. MobileViT-S, a su vez, superó a Swin-T por 0,6 puntos. Sobre este mismo *test*, la diferencia entre MobileViT-S y ViT-B/16 fue similar en los tres niveles de dificultad: 1,9 puntos en clases fáciles, 2,4 en medias y 2,6 en difíciles.

7. Ejemplos y visualizaciones

7.1 Curvas de validación

Para visualizar la evolución del entrenamiento se utilizaron dos figuras generadas con *4.0-comparativa.ipynb* a partir de los historiales de entrenamiento. La primera muestra el F1
Visión por Computadora III - Food-101 benchmark

macro de validación por época para las distintas configuraciones probadas en cada arquitectura, y permite observar los ajustes analizados en la sección 6.4. La segunda compara las configuraciones finalmente seleccionadas: ViT-B/16 con *learning rate* 5×10^{-5} , y MobileViT-S, Swin-T y DeiT-Ti con la configuración común. Las curvas se muestran en F1 macro porque fue la métrica de selección del *checkpoint* (sección 5.1); la *accuracy* de validación tiene la misma forma.

Efecto de la receta dentro de cada arquitectura

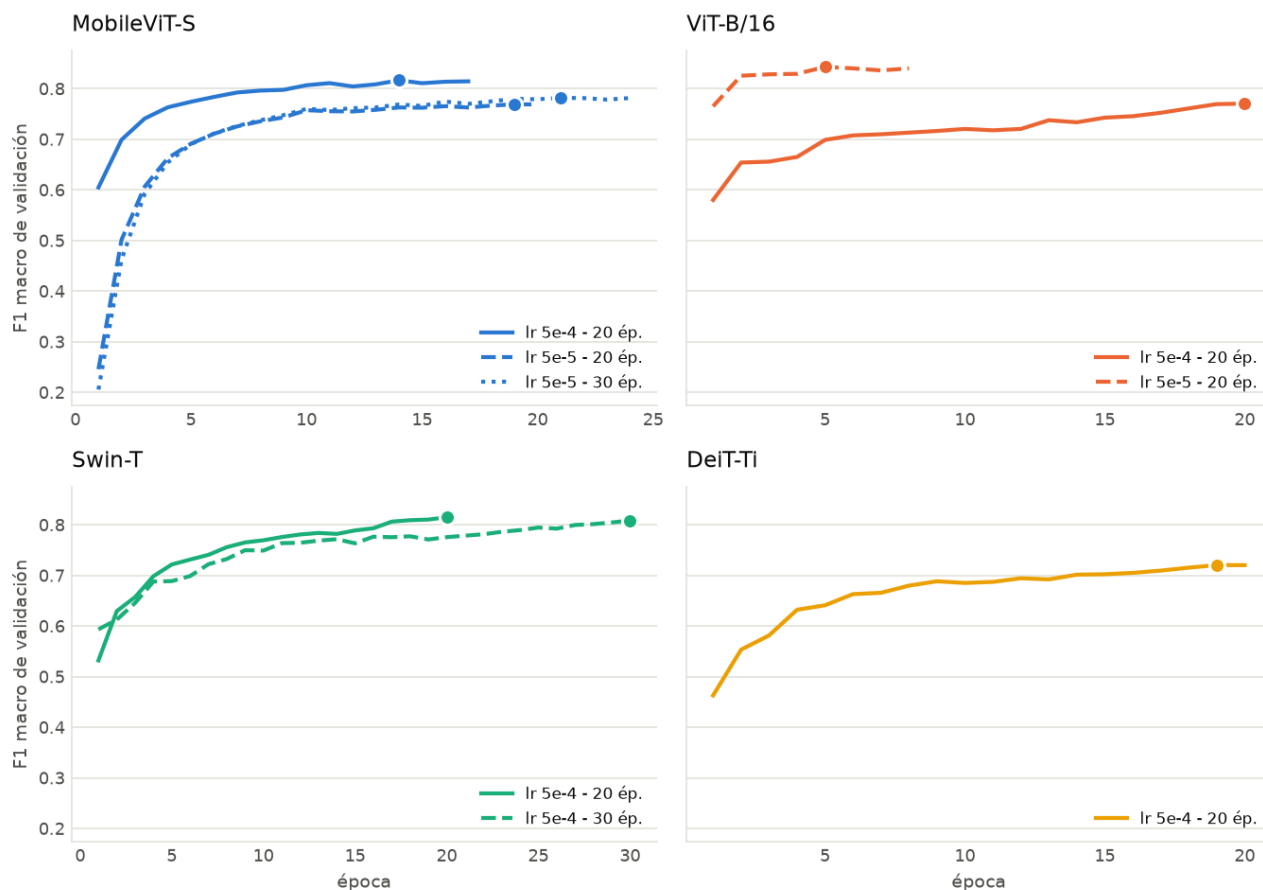


Figura 5. F1 macro de validación por época para las configuraciones probadas en cada arquitectura. El punto indica la época del mejor *checkpoint*.

Convergencia del entrenamiento

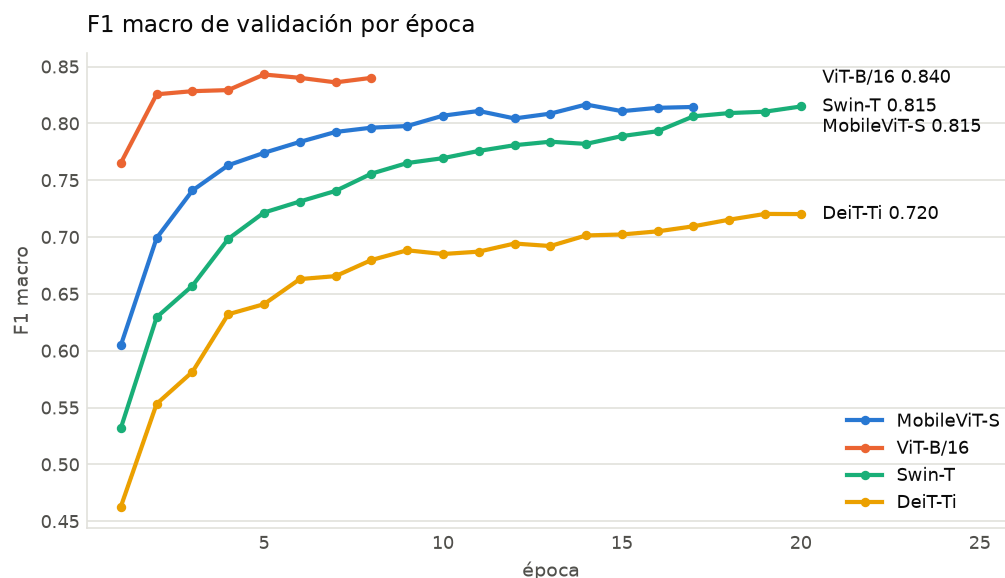


Figura 6. F1 macro de validación por época de las configuraciones seleccionadas de cada arquitectura.

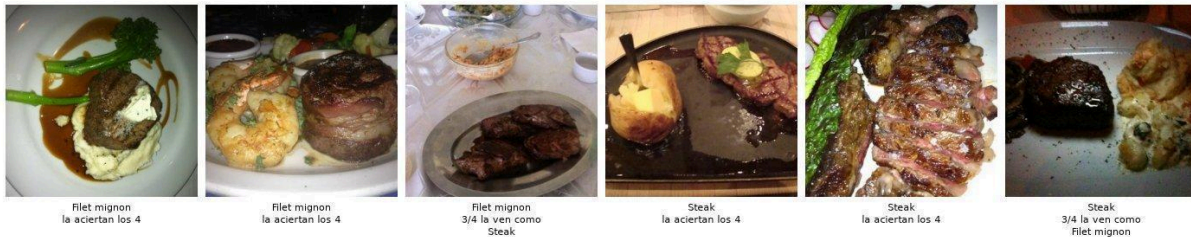
En la Figura 5 se observa que el cambio de *learning rate* tuvo efectos diferentes según la arquitectura. En ViT-B/16, reducirlo a 5×10^{-5} mejoró el desempeño de validación y el mejor valor se alcanzó en la época 5. En MobileViT-S, en cambio, el menor *learning rate* produjo peores resultados y no superó a la configuración original ni al extender el entrenamiento a 30 épocas. En Swin-T, ambas corridas alcanzaron resultados muy similares. En la Figura 6, ViT-B/16 ajustado alcanza rápidamente el mayor valor de validación, mientras que MobileViT-S y Swin-T evolucionan de forma más gradual y DeiT-Ti se mantiene por debajo. La curva de ViT-B/16 termina en la época 8 debido al *early stopping*.

7.2 Ejemplos de predicciones y errores

Para complementar los resultados cuantitativos, se muestran ejemplos de imágenes clasificadas correctamente y de errores cometidos por los modelos. Las grillas se generaron con *4.0-comparativa.ipynb* a partir de las predicciones de las configuraciones seleccionadas de cada arquitectura (sección 6.4). Cada imagen se muestra recortada al cuadrado central, tal como la ve el modelo.

Pares de clases que más se confunden

Filet mignon ↔ Steak · 330 confusiones entre los cuatro modelos



Chocolate cake ↔ Chocolate mousse · 218 confusiones entre los cuatro modelos



Beef tartare ↔ Tuna tartare · 157 confusiones entre los cuatro modelos



Pork chop ↔ Steak · 151 confusiones entre los cuatro modelos



Figura 7. Pares de clases que más se confunden entre los cuatro modelos. Por cada clase, dos imágenes que los cuatro aciertan y una que al menos tres asignan a la otra clase del par.

Steak, *filet mignon* y *pork chop* presentan una apariencia visual muy similar: carne cocida en un plato, con guarnición y salsa. Lo que las distingue es el corte y el grosor, que desde el ángulo de la foto muchas veces no se ve. Entre *chocolate cake* y *chocolate mousse* la principal diferencia visual está en la textura, mientras que los dos tipos de *tartare* suelen presentar una apariencia y una forma de presentación similares: en torre con hojas verdes encima. Estas confusiones se repiten en las cuatro arquitecturas y forman parte de los errores compartidos analizados en la sección 6.3.

Imágenes que acierta MobileViT-S y falla ViT-B/16

Tuna tartare • F1 0.82 (MobileViT-S) contra 0.76 (ViT-B/16)



Baby back ribs • F1 0.85 (MobileViT-S) contra 0.82 (ViT-B/16)



Pad thai • F1 0.93 (MobileViT-S) contra 0.91 (ViT-B/16)



Fried rice • F1 0.88 (MobileViT-S) contra 0.86 (ViT-B/16)



Figura 8. Clases donde MobileViT-S más supera a ViT-B/16. Cada imagen la acierta MobileViT-S y la falla ViT-B/16; el epígrafe indica la predicción de ViT-B/16.

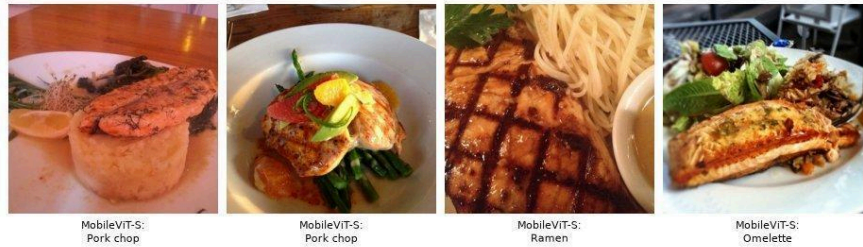
MobileViT-S supera a ViT-B/16 en 19 de las 101 clases, aunque en general las diferencias son pequeñas. Las mayores se observan en *tuna tartare*, *baby back ribs*, *pad thai* y *fried rice*. En estas clases, algunos errores de ViT-B/16 corresponden a platos visualmente similares, como *foie gras*, *beet salad* o *beef carpaccio* para *tuna tartare*, *pork chop* para *baby back ribs* y *fried rice* para *pad thai*.

Imágenes que acierta ViT-B/16 y falla MobileViT-S

Apple pie • F1 0.67 (MobileViT-S) contra 0.76 (ViT-B/16)



Grilled salmon • F1 0.78 (MobileViT-S) contra 0.87 (ViT-B/16)



Breakfast burrito • F1 0.76 (MobileViT-S) contra 0.84 (ViT-B/16)



Gnocchi • F1 0.79 (MobileViT-S) contra 0.86 (ViT-B/16)



Figura 9. Clases donde ViT-B/16 más supera a MobileViT-S. Cada imagen la acierta ViT-B/16 y la falla MobileViT-S; el epígrafe indica la predicción de MobileViT-S.

En el otro sentido la brecha es mayor: ViT-B/16 supera a MobileViT-S en 82 clases, y las diferencias más grandes se dan en *apple pie*, *grilled salmon*, *breakfast burrito* y *gnocchi*. En estas imágenes MobileViT-S confunde platos que comparten color y textura con otra clase: una tarta de cobertura dorada con *baklava*, un salmón con guarnición con *pork chop*, *gnocchi* con *ravioli*. En estos ejemplos, ViT-B/16, que tiene aproximadamente 17 veces más parámetros, logra distinguir mejor estos detalles visuales.

Imágenes que los cuatro modelos fallan del mismo modo

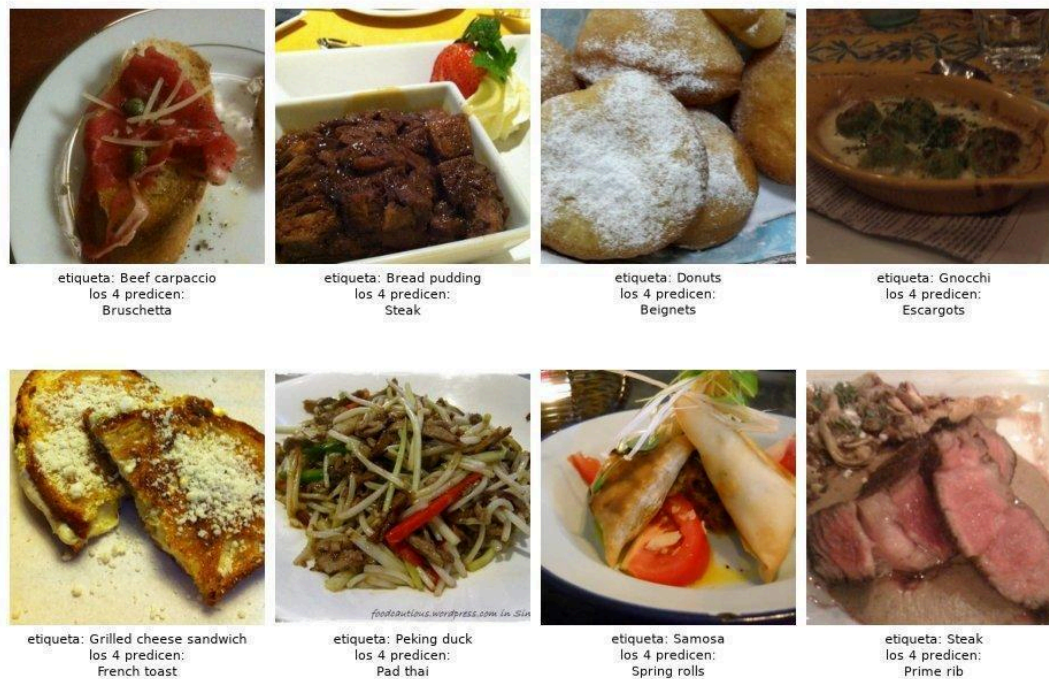


Figura 10. Imágenes en las que los cuatro modelos fallan prediciendo la misma clase equivocada.

En varios de estos casos la predicción común es defendible: un *beef carpaccio* servido sobre pan es visualmente una *bruschetta*, los *donuts* con azúcar impalpable parecen *beignets*, un *grilled cheese* espolvoreado parece *french toast* y una porción de *steak* fileteada parece *prime rib*. Cuando las cuatro arquitecturas coinciden en el mismo error, esto sugiere que existe cierta ambigüedad en la imagen o en la etiqueta, y ayuda a explicar parte de los errores compartidos observados.

8. Conclusiones

Los resultados muestran que cada arquitectura responde de manera diferente a la configuración de entrenamiento. ViT-B/16 mejoró mucho al reducir el *learning rate*, mientras que MobileViT-S obtuvo mejores resultados con la configuración original. Esto indica que usar una misma configuración para todos los modelos sirve como punto de partida, pero no necesariamente permite obtener el mejor rendimiento de cada uno.

Entre las mejores configuraciones evaluadas, ViT-B/16 logró el mayor desempeño de clasificación. Sin embargo, la diferencia respecto de MobileViT-S fue pequeña y similar en los tres niveles de dificultad de las clases: la hipótesis de que la capacidad extra se notaría sobre todo en las clases más difíciles de distinguir no se confirmó para MobileViT-S ni para Swin-T, y sí para DeiT-Ti. Swin-T obtuvo resultados cercanos a MobileViT-S, aunque con un costo computacional mayor, mientras que DeiT-Ti presentó el menor costo de inferencia, pero también el menor desempeño de clasificación.

Para el objetivo planteado en este trabajo, MobileViT-S mostró una relación muy favorable entre desempeño y eficiencia. Conserva más del 97 % de la *accuracy* de ViT-B/16 utilizando aproximadamente 17 veces menos parámetros y casi 9 veces menos GFLOPs, y su latencia en CPU con cuantización INT8 es unas 2,6 veces menor. MobileViT-S y ViT-B/16 representan los mejores compromisos entre desempeño y costo: no hay otro modelo que los supere en rendimiento utilizando al mismo tiempo menos recursos. Estos resultados posicionan a MobileViT-S como un buen candidato para entornos con recursos limitados, mientras que ViT-B/16 alcanza el mayor desempeño de clasificación entre las configuraciones evaluadas.

9. Trabajo futuro

A partir de los resultados obtenidos, quedan algunas líneas de trabajo que podrían ampliar la evaluación realizada:

- Ajuste de entrenamiento: explorar configuraciones específicas para Swin-T y DeiT-Ti, especialmente distintos valores de *learning rate*, ya que estas arquitecturas no fueron ajustadas con el mismo nivel de detalle que ViT-B/16 y MobileViT-S.
- Regularización: evaluar estrategias como *label smoothing* o una política de *augmentation* más fuerte para analizar si permiten mejorar la generalización de los modelos.

10. Planificación del equipo

Tarea	Responsable	Estado
Búsqueda y evaluación de datasets candidatos	Equipo completo	Completado
Estructura inicial del proyecto para la primera alternativa de plantas	Flor	Completado — línea posteriormente descartada
EDA de Cassava y análisis de viabilidad del dataset	Antonella	Completado — dataset descartado
EDA de alternativa PlantSeg	Flor	Completado — línea posteriormente descartada
Cambio de enfoque y selección de Food-101	Equipo completo	Completado
Estructura inicial del proyecto Food-101	Marcos	Completado
EDA inicial de Food-101 y generación de artefactos del benchmark	Marcos	Completado
Preprocesamiento: splits, cache, transforms, DataLoaders y validación contra AutoImageProcessor	Marcos	Completado
Primera prueba de MobileViT: smoke test y entrenamiento con dataset completo	Antonella	Completado
Incorporación inicial de Swin-T, DeiT-Ti y ViT-B/16 al benchmark	Flor	Completado
Modularización del pipeline común de entrenamiento, evaluación y benchmark	Flor	Completado

Tarea	Responsable	Estado
Evaluación de eficiencia: tamaños FP32/FP16/INT8 y latencia con cuantización INT8	Flor	Completado
Corridas base definitivas de MobileViT-S, DeiT-Ti, Swin-T y ViT-B/16	Antonella	Completado
Curvas de entrenamiento y registro con TensorBoard	Antonella	Completado
Integración y registro de resultados con MLflow	Antonella	Completado
Notebook comparativo de los cuatro modelos	Marcos	Completado
Análisis comparativo: desempeño/costo, dificultad, errores compartidos y convergencia	Marcos	Completado
Generación de figuras y ejemplos para el análisis comparativo	Marcos	Completado
Experimentos adicionales con learning rate y número de épocas	Antonella	Completado
Informe técnico final	Antonella + revisión del equipo	Completado
Presentación final	Equipo completo	Pendiente

11. Referencias

Bossard, L., Guillaumin, M., & Van Gool, L. (2014). *Food-101: Mining Discriminative Components with Random Forests*. ECCV 2014.

Hugging Face. *Image classification: Transformers Documentation*. Tutorial de fine-tuning de ViT sobre Food-101. https://huggingface.co/docs/transformers/tasks/image_classification

Dosovitskiy, A., Beyer, L., Kolesnikov, A., Weissenborn, D., Zhai, X., Unterthiner, T., Dehghani, M., Minderer, M., Heigold, G., Gelly, S., Uszkoreit, J., & Hounsby, N. (2021). *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. ICLR 2021.

Touvron, H., Cord, M., Douze, M., Massa, F., Sablayrolles, A., & Jégou, H. (2021). *Training data-efficient image transformers & distillation through attention*. ICML 2021.

Liu, Z., Lin, Y., Cao, Y., Hu, H., Wei, Y., Zhang, Z., Lin, S., & Guo, B. (2021). *Swin Transformer: Hierarchical Vision Transformer using Shifted Windows*. ICCV 2021.

Mehta, S., & Rastegari, M. (2022). *MobileViT: Light-weight, General-purpose, and Mobile-friendly Vision Transformer*. ICLR 2022.

Código y artefactos del proyecto: [marcoslund/ViT-for-101-food-app](https://github.com/marcoslund/ViT-for-101-food-app)